

Document RAG API - IAM

AccessDenied障害対応Runbook

1. 目的

本Runbookは、Document RAG APIからAWSサービスへアクセスする際に、AccessDeniedまたは権限不足に関連するエラーが発生した場合の確認および切り分け手順を定める。

対象となる主な事象は以下とする。

- Amazon Bedrockのモデル呼び出し時にAccessDeniedが発生する
- Amazon S3へのアクセス時にAccessDeniedが発生する
- Amazon S3 Vectorsへのアクセス時にAccessDeniedが発生する
- ECS Task Roleの権限不足が疑われる
- ECS Task Execution RoleとTask Roleの設定先を誤っている

2. IAM障害対応の基本フロー

AccessDeniedが発生した場合は、以下の順序で確認する。

1. CloudWatch LogsでAccessDeniedExceptionの内容を確認する
2. エラーに記録されているActionを確認する
3. エラーに記録されているResource ARNを確認する
4. AWS APIを実行したPrincipalを確認する
5. 対象Principalに付与されているIAM Policyを確認する
6. PolicyのActionおよびResourceが対象APIを許可しているか確認する

原因がIAM Policyの不足である場合でも、Full Access Policyを付与して解決してはならない。

必要なActionおよびResourceを特定し、最小権限となるよう権限を追加する。

3. ECS Task RoleとTask Execution Role

Document RAG APIでは、ECS Task RoleとECS Task Execution Roleを異なる目的で使用する。

ECS Task Role

ECS上で動作するFastAPIアプリケーションがAWS APIを呼び出すために使用する。

Document RAG APIでは主に以下へのアクセス権限を持つ。

- Amazon Bedrock
- Amazon S3
- Amazon S3 Vectors
- Amazon Textract

アプリケーション実行中にBedrockやS3へのAccessDeniedが発生した場合は、まずECS Task Roleを確認する。

ECS Task Execution Role

ECSがコンテナを起動・実行するために使用する。

主に以下の処理で利用される。

- Amazon ECRからDocker Imageを取得する
- CloudWatch Logsへコンテナログを送信する

アプリケーションからBedrockやS3へアクセスする権限をTask Execution Roleへ追加してはならない。

4. AccessDeniedExceptionの確認

CloudWatch LogsでAccessDeniedExceptionを確認した場合、エラーメッセージから以下を特定する。

- Principal
- Action
- Resource

例えば、S3 Vectorsへのアクセスで以下のActionが拒否されている場合を考える。

s3vectors:GetVectors

この場合は、ECS Task Roleに付与されたS3 Vectors用IAM Policyで s3vectors:GetVectors が許可されているか確認する。

Actionが許可されていても、PolicyのResourceが実際にアクセスしているVector BucketまたはIndexを対象としていなければAccessDeniedとなる可能性がある。

5. Amazon BedrockのAccessDenied

Amazon Bedrockのモデル呼び出しでAccessDeniedが発生した場合は、使用しているAPIとModel IDを確認する。

Document RAG APIでは、LLM回答生成とEmbedding生成で異なるモデルを使用する。

LLM回答生成

Claude Sonnet 4.6を利用する。

Inference Profileを利用している場合は、Inference Profileおよび対象Foundation Modelへのアクセス権限を確認する。

主な確認対象Actionは以下とする。

- bedrock:InvokeModel

Embedding生成

Amazon Titan Text Embeddings V2を利用する。

Embedding生成時にAccessDeniedが発生した場合は、Titan EmbeddingsのFoundation Modelに対して bedrock:InvokeModel が許可されているか確認する。

6. Amazon S3のAccessDenied

Document RAG APIでは、PDF本体およびDocument MetadataをAmazon S3へ保存する。

S3でAccessDeniedが発生した場合は、実行しようとしている処理に応じて以下のActionを確認する。

- s3:PutObject
- s3:GetObject
- s3:DeleteObject

- s3:ListBucket

Object操作ではObject ARNを対象Resourceとして指定する。

Bucket一覧取得などBucket自体に対する操作ではBucket ARNを対象Resourceとして指定する。

Actionが正しくても、Bucket ARNとObject ARNを誤って設定するとアクセスできない場合があるため注意する。

7. Amazon S3 VectorsのAccessDenied

Document RAG APIでは、チャンクおよびEmbeddingをAmazon S3 Vectorsへ保存する。

主な利用Actionは以下とする。

- s3vectors:PutVectors
- s3vectors:QueryVectors
- s3vectors:GetVectors
- s3vectors>DeleteVectors

RAG検索時にAccessDeniedが発生した場合は、Queryに必要なActionだけでなく、処理の中で利用している関連ActionがIAM Policyに含まれているか確認する。

特にエラーログに具体的なAction名が表示されている場合は、そのActionがTask Roleで許可されているかを確認する。

8. 最小権限の原則

AccessDeniedを解消する目的で以下の対応を行ってはならない。

- Action: "*" を設定する
- AdministratorAccess を付与する
- 不要なFull Access Policyを付与する

必要なActionを特定したうえで、可能な限りResourceも対象リソースへ限定する。

例えばDocument RAG APIのS3アクセス権限では、アプリケーションが使用するDocument Bucketのみを対象とする。

権限追加後は、対象APIが正常に実行できることを確認する。

9. 復旧確認

IAM Policy変更後は以下を確認する。

1. AccessDeniedが発生していたAPIを再実行する
2. HTTPレスポンスが正常に返ることを確認する
3. CloudWatch Logsに同じAccessDeniedExceptionが再発していないことを確認する
4. 必要以上の権限を追加していないことを確認する

例えば /chat でS3 VectorsのAccessDeniedが発生していた場合は、IAM Policy変更後に /chat を再実行し、RAG回答が正常に返ることを確認する。

10. エスカレーション

以下の場合はクラウド基盤担当者へエスカレーションする。

- 必要なActionまたはResourceを特定できない
- Policy上は許可されているにもかかわらずAccessDeniedが継続する
- Inference Profileなど複数Resourceにまたがる権限設定が必要
- IAM Policy以外のアクセス制御が原因として疑われる
- 権限変更による影響範囲を判断できない

エスカレーション時には以下を共有する。

- AccessDeniedExceptionのエラーメッセージ
- Principal
- Action
- Resource ARN
- 対象のECS Task Role
- 現在設定されているIAM Policy
- 発生しているAPIまたは処理